DELTA – Střední škola informatiky a ekonomie, s.r.o.
Ke Kamenci 151, Pardubice

LifeMeter

Mobilní aplikace pro sledování zdravotních metrik

Autor: Zadrobílek Jan
Třída: 4.B
Studijní obor: Informační technologie 18-20-M/01
Školní rok: 2025/2026

Zadání maturitního projektu z informatických předmětů

Jméno a příjmení:	*Jan Zadrobílek*
Školní rok:	*2025/2026*
Třída:	*4.B*
Obor:	*Informační technologie 18-20-M/01*
Téma práce:	*Mobilní aplikace „LifeMeter“*
Vedoucí práce:	*Ing. Monika Borkovcová, Ph.D.*

Způsob zpracování, cíle práce, pokyny k obsahu a rozsahu práce:

Cílem projektu je vytvoření mobilní aplikace pro OS Android, která bude sloužit jako nástroj pro sledování fyzické aktivity, nutričních hodnot a spánku. Funkce budou zahrnovat možnost přihlášení, notifikace a lokální manipulaci s daty. Aplikace bude navržena tak, aby poskytovala uživatelům intuitivní uživatelské rozhraní a efektivní způsob interakce. Aplikace je funkční pro OS Android 10 a vyšší.

Popis funkčnosti mobilní aplikace, která umožňuje:

Záznam fyzických aktivit

- Možnost přidávat aktivity s informacemi jako poloha nebo poznámka.

Sledování výživy

- Přidávání zkonzumovaného jídla, doplňků stravy, sledování kalorií, makroživin a vitamínů.

Sledování spánku

- Záznam délky spánku s možností přidání poznámek, případně možnost pracovat s kvalitou spánku.

Správa uživatelů

- Přihlašování bude probíhat pomocí emailu se zabezpečeným ověřením.

Offline

- Aplikace bude umožňovat offline režim se synchronizací dat při obnovení připojení.

Notifikace

- Upozornění ke cvičení a spánku.

Integrace s jinými aplikacemi

- Aplikace bude umožňovat propojení s vybranou aplikací/platformou.

Prohlašuji, že jsem maturitní projekt vypracoval(a) samostatně, výhradně s použitím uvedené literatury.

V Pardubicích 11. 3. 2026 .....

(vlastnoruční podpis)

Rád bych upřímně poděkoval své vedoucí práce, Ing. Monice Borkovcové, za odborné vedení, rady, trpělivost a čas, který mi věnovala při konzultacích a zpracování tohoto maturitního projektu.

Resumé

Tato práce představuje aplikaci LifeMeter, určenou pro sledování a analýzu životních metrik uživatele za účelem osobního rozvoje a zlepšení zdraví. Aplikace umožňuje uživatelům centralizované zaznamenávání a monitorování stravy, kvality spánku a fyzické aktivity.

Backendová část systému je postavena na technologiích Bun, Hono a relační databázi PostgreSQL. Pro komunikaci mezi mobilní aplikací a serverem je implementováno REST API, které zajišťuje autentizaci, synchronizaci dat a správu uživatelských profilů.

Mobilní klientská aplikace je vyvinuta pomocí technologií React Native a Expo. Zvolený přístup zaručuje kompatibilitu a zároveň umožňuje přístup k nativním rozhraním pro synchronizaci zdravotních dat, konkrétně HealthKit pro operační systém iOS a Health Connect pro platformu Android.

Klíčová slova: *mobilní aplikace, zdravotní metriky, React Native, Expo, REST API, Bun, Hono, PostgreSQL*

Summary

This thesis presents the LifeMeter application, designed for tracking and analyzing a user's lifestyle metrics to support personal development and health improvement. The application allows users to centrally log and monitor their diet, sleep quality, and physical activity.

The backend of the system is built on Bun, Hono, and a PostgreSQL relational database. A REST API is implemented for communication between the mobile application and the server, providing authentication, data synchronization, and user profile management.

The mobile client application is developed using React Native and Expo. The chosen approach ensures cross-platform compatibility while enabling access to native APIs for health data synchronization, specifically HealthKit for the iOS operating system and Health Connect for Android.

Keywords: *mobile application, health metrics, React Native, Expo, REST API, Bun, Hono, PostgreSQL*

Obsah

Úvod **7**
0.1 Cíl práce . . . 7
0.2 Struktura práce . . . 7

1 Teoretická část **8**
1.1 Mobilní platformy a zdravotní data . . . 8
1.1.1 HealthKit (iOS) . . . 8
1.1.2 Health Connect (Android) . . . 8
1.2 Mobilní vývoj s React Native a Expo . . . 8
1.2.1 React Native . . . 8
1.2.2 Expo . . . 8
1.3 Serverová architektura . . . 8
1.3.1 REST API . . . 8
1.3.2 Autentizace . . . 8

2 Architektura systému **9**
2.1 Architektonický vzor . . . 9
2.1.1 Popis vrstev aplikace . . . 10
2.2 Přehled komponent systému . . . 12
2.3 Use Case diagram . . . 15
2.3.1 Aktéři . . . 15
2.3.2 Případy užití . . . 15
2.4 Diagram tříd . . . 15
2.4.1 Popis klíčových tříd . . . 15
2.5 Datový model . . . 15
2.5.1 Schéma databáze . . . 15
2.5.2 Popis tabulek . . . 15
2.6 Offline režim a synchronizace . . . 15
2.7 Bezpečnost a autentizace . . . 15

3 Metodika a vlastní řešení **16**
3.1 Mobilní klient . . . 16
3.1.1 Knihovna pro zdravotní data (`health.ts`) . . . 16
3.1.2 Offline fronta požadavků . . . 16
3.1.3 Autentizace a perzistence stavu . . . 16
3.1.4 Notifikace . . . 16
3.2 Backend API . . . 16
3.2.1 Technologie a nasazení . . . 16
3.2.2 CI/CD pipeline . . . 16
3.2.3 Dokumentace endpointů . . . 16

4 Výsledky **17**
4.1 Funkční části aplikace . . . 17
4.1.1 Mobilní klient . . . 17
4.1.2 Backend . . . 17
4.2 Testování . . . 17
4.2.1 Testování na fyzickém zařízení . . . 17
4.2.2 Synchronizace zdravotních dat . . . 17

5 Diskuse **18**
5.1 Zhodnocení architektonických voleb . . . 18
5.1.1 React Native vs. Flutter . . . 18
5.1.2 Vlastní backend vs. Supabase . . . 18
5.2 Otevřené problémy . . . 18
5.2.1 30denní limit Health Connect . . . 18
5.3 Možná rozšíření . . . 18

Závěr **19**
5.4 Shrnutí . . . 19
5.5 Podmínky provozu . . . 19
5.6 Budoucí rozvoj . . . 19

Literatura **20**

Seznam obrázků **21**

Seznam tabulek **22**

Seznam příloh **23**

Úvod

Zdravý životní styl a sledování osobních návyků hrají v dnešní době důležitou roli. Na trhu sice existuje množství aplikací pro fitness, výživu či spánek, ale nutnost používat více nástrojů současně bývá nepřehledná a demotivující. Proto vznikla myšlenka mobilní aplikace „LifeMeter“, která tyto klíčové aspekty sjednocuje do jednoho uživatelsky přívětivého prostředí.

0.1 Cíl práce

Cílem maturitního projektu je návrh a vývoj aplikace „LifeMeter“ pro operační systém Android (verze 10 a vyšší). Požadované funkce zahrnují:

- **Záznam fyzických aktivit:** Možnost přidávat aktivity s informacemi jako poloha nebo poznámka.
- **Sledování výživy:** Přidávání zkonzumovaného jídla, doplňků stravy, sledování kalorií, makroživin a vitamínů.
- **Sledování spánku:** Záznam délky spánku s možností přidání poznámek, případně možnost pracovat s kvalitou spánku.
- **Správa uživatelů:** Přihlašování bude probíhat pomocí emailu se zabezpečeným ověřením.
- **Offline:** Aplikace bude umožňovat offline režim se synchronizací dat při obnovení připojení.
- **Notifikace:** Upozornění ke cvičení a spánku.
- **Integrace s jinými aplikacemi:** Aplikace bude umožňovat propojení s vybranou aplikací/platformou.

0.2 Struktura práce

Text práce je rozdělen do navazujících částí.

1. Teoretická část

1.1 Mobilní platformy a zdravotní data

1.1.1 HealthKit (iOS)

1.1.2 Health Connect (Android)

1.2 Mobilní vývoj s React Native a Expo

1.2.1 React Native

1.2.2 Expo

1.3 Serverová architektura

1.3.1 REST API

1.3.2 Autentizace

2. Architektura systému

2.1 Architektonický vzor

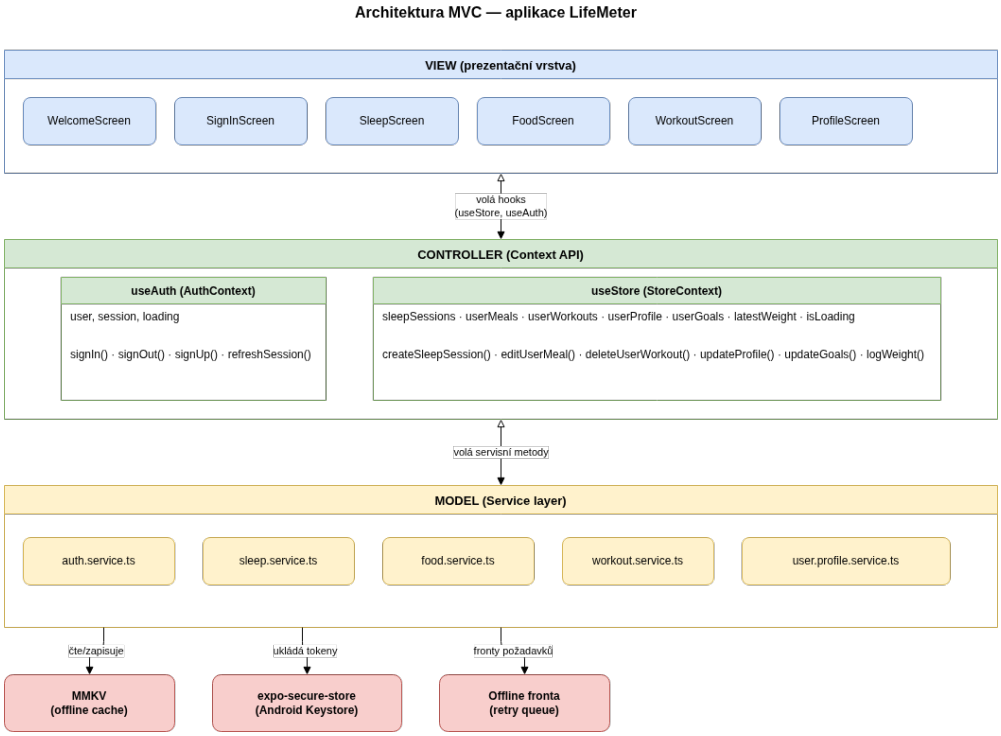
Mobilní aplikace LifeMeter je vytvořena v prostředí React Native s využitím platformy Expo. Namísto nativního vývoje pro Android byl zvolen tento technologický stack zejména z důvodu potenciální budoucí kompatibility s platformou iOS, přičemž přechod na nativní aplikaci by vyžadoval zásadní přepsání kódové základny. Dalším faktorem bylo, že vývoj v jazyce TypeScript a frameworku React umožnil výrazně efektivnější implementaci oproti nativnímu Androidu s využitím jazyka Kotlin, s nímž nebyly k dispozici dostatečné předchozí zkušenosti.

Architektura aplikace vychází z návrhového vzoru **MVC** (Model–View–Controller) [1], který je přizpůsoben prostředí React Native. Jednotlivé vrstvy odpovídají následujícímu rozdělení zodpovědností:

- **Model:** Servisní třídy (`sleep.service.ts`, `food.service.ts`, ...) zapouzdřují veškerou komunikaci se vzdáleným API serverem. Každá třída odpovídá jedné doménové oblasti a poskytuje typované metody pro CRUD operace nad příslušnými zdroji.

- **View:** Obrazovky a komponenty implementované v souborech `.tsx` tvoří prezentační vrstvu. Komponenty jsou čistě zobrazovací; žádná business logika ani přímá komunikace se serverem v nich není přítomna.

- **Controller:** Globální stavové kontexty `useStore` a `useAuth` plní roli controlleru. Kontext `useAuth` řídí stav autentizace a životní cyklus relace, zatímco `useStore` agreguje data všech domén (spánek, jídlo, tréninky, profil) a vystavuje akce, které komponenty volají prostřednictvím hooků React.



Obrázek 2.1: Schéma architektonického vzoru MVC aplikace LifeMeter

2.1.1 Popis vrstev aplikace

Prezentační vrstva (View)

Prezentační vrstva se skládá z obrazovek organizovaných do adresářů podle funkčních oblastí (`sleep`, `food`, `workout`, `onboarding`, `profile`). Navigace mezi obrazovkami je zajištěna knihovnou `@react-navigation` prostřednictvím nativního zásobníku obrazovek (`createNativeStackNavigator`) a spodní lišty karet (`createBottomTabNavigator`). Komponenty jsou stylizovány pomocí knihovny Tailwind CSS adaptované pro React Native prostřednictvím balíčku `uniwind` a komponentové knihovny `heroui-native`.

Vrstva controlleru (Context)

Stav aplikace je centrálně spravován dvěma kontexty poskytovanými React Context API:

- `AuthContext` (`useAuth`) — uchovává referenci na přihlášeného uživatele a aktivní relaci, zprostředkovává operace přihlášení, odhlášení a registrace a reaguje na výsledek volání `auth.service.ts`.
- `StoreContext` (`useStore`) — drží v paměti kolekce dat pro všechny domény (záznamy spánku, jídla, tréninků, profil, cíle, váhové záznamy) a vystavuje *action callbacks* (`createSleepSession`, `editUserMeal`, `deleteUserWorkout` aj.). Počáteční

načtení dat probíhá při montáži kontextu pomocí paralelních volání (`Promise.all`); aktualizace stavu po mutacích jsou prováděny optimisticky přímo v paměti.

Datová vrstva (Model / Services)

Datová vrstva tvoří rozhraní mezi aplikací a vzdáleným REST API serverem na adrese `https://lifemeter.fit`. Každá servisní třída komunikuje se serverem prostřednictvím pomocných funkcí `request()` a `fetchWithTimeout()`, které obalují nativní `fetch` API a zajišťují jednotné zpracování chyb a časových limitů.

Autentizace je řešena prostřednictvím knihovny Better Auth s relacemi na bázi cookies (`credentials: 'include'`). Citlivé uživatelské údaje (například přihlašovací token) jsou ukládány do zabezpečeného úložiště platformy pomocí `expo-secure-store`, které na Androidu využívá Android Keystore, respektive iOS Secure Enclave na platformě iOS.

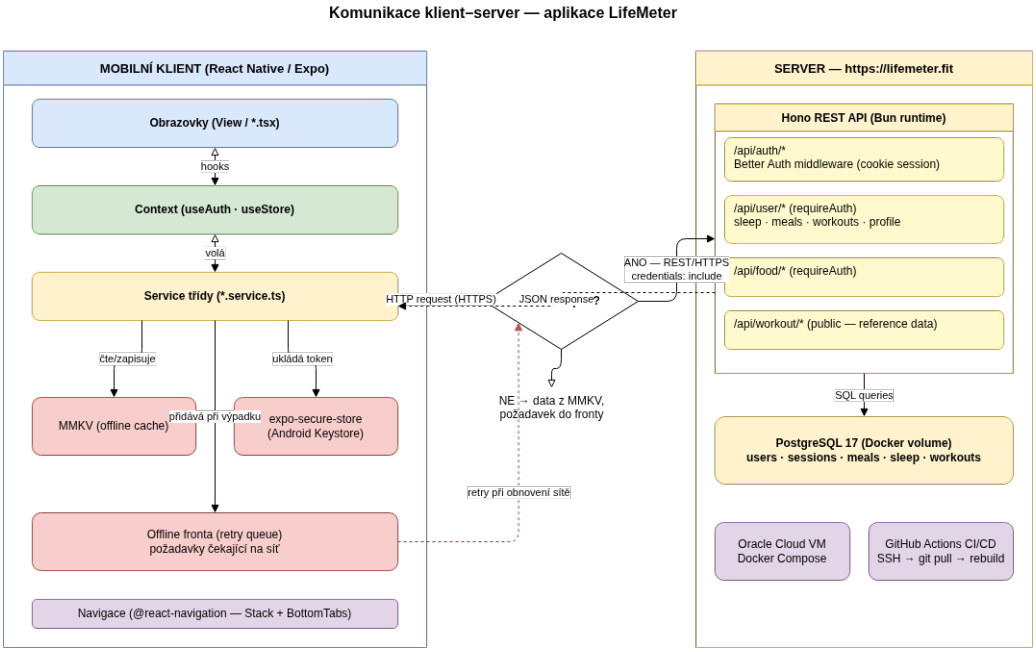
Offline vrstva a fronta požadavků [2]

Aplikace podporuje offline režim prostřednictvím lokálního úložiště **MMKV**, jež poskytuje vysokorychlostní synchronní přístup k perzistentním datům přímo z vlákna JavaScriptu bez blokování hlavního vlákna UI. Při startu aplikace jsou data načítána ze serveru; v případě nedostupnosti sítě jsou zobrazena lokálně uložená data.

Operace, jejichž provedení selhalo z důvodu nedostupnosti připojení, jsou zařazeny do **offline fronty**. Po obnovení konektivity jsou požadavky z fronty automaticky opětovně odeslány, čímž je zajištěna konzistence dat mezi lokálním stavem aplikace a serverem.

Backend

Serverová část aplikace je tvořena REST API postaveným na frameworku **Hono** s runtime prostředím **Bun**. Databáze je provozována na PostgreSQL 17. Server je trvale nasazen na adrese `https://lifemeter.fit` prostřednictvím Docker Compose na virtuálním stroji Oracle Cloud. Nasazení nové verze je automatizováno prostřednictvím GitHub Actions, které při každém pushnutí do větve `main` provede aktualizaci serveru přes SSH. Přehled komunikace mezi klientem a serverem je znázorněn na obrázku 2.2.



Obrázek 2.2: Komunikace mezi klientem a serverem aplikace LifeMeter

2.2 Přehled komponent systému

Systém LifeMeter se skládá ze čtyř hlavních skupin komponent: mobilního klienta, serverové části, externích zdravotních platforem a lokálních úložišť na straně zařízení. Jejich vzájemné vztahy jsou znázorněny na obrázku 2.3.

Obrázek 2.3: Přehled komponent systému LifeMeter

Mobilní klient

Mobilní klient je implementován v prostředí React Native s platformou Expo a tvoří jedinou vstupní bránu, skrze níž uživatel interaguje s celým systémem. Klient zajišťuje prezentaci dat, správu globálního stavu prostřednictvím kontextů `AuthContext` a `StoreContext`, navigaci mezi obrazovkami a koordinaci všech ostatních komponent popsaných níže.

REST API server

Serverová část je postavena na frameworku **Hono** s runtime prostředím **Bun** a je dostupná na adrese `https://lifemeter.fit`. Server plní roli centrálního bodu zpracování business logiky: přijímá požadavky od mobilního klienta, ověřuje autentizaci, provádí validaci vstupních dat a zprostředkovává přístup k databázi. Veškerá komunikace mezi klientem a serverem probíhá prostřednictvím protokolu HTTPS.

PostgreSQL 17

Primárním datovým úložištěm na straně serveru je relační databáze **PostgreSQL 17**. Jsou v ní uloženy veškeré uživatelské záznamy (spánek, jídlo, tréninky, profil, cíle, váhové záznamy), referenční číselník potravin a data správy relací spravovaná knihovnou Better Auth. Přístup k databázi probíhá výhradně přes API server; klient k ní nemá přímý přístup.

MMKV — lokální úložiště

Na straně klienta slouží knihovna **MMKV** jako vysokorychlostní perzistentní cache serverových dat. Data jsou do MMKV zapsána po každém úspěšném načtení ze serveru a při nedostupnosti sítě jsou z ní čtena, čímž je zajištěn offline přístup k posledním známým datům. MMKV rovněž uchovává serializovanou frontu neodoslaných mutačních požadavků (viz sekci 2.6).

expo-secure-store

Citlivé přihlašovací údaje, zejména session token vydaný serverem, jsou ukládány prostřednictvím knihovny **expo-secure-store**. Ta na platformě Android využívá systémové úložiště Android Keystore, na platformě iOS pak Secure Enclave. Díky tomu nejsou citlivé údaje nikdy ukládány do nezabezpečených úložišť, jako jsou MMKV nebo AsyncStorage.

Health Connect a Apple HealthKit

Za účelem rozšíření zdrojů zdravotních dat jsou integrovány dvě platformní zdravotní služby. Na platformě Android je využíváno **Health Connect**, standardizované rozhraní systému Android pro výměnu zdravotních dat mezi aplikacemi. Na platformě iOS plní ekvivalentní roli **Apple HealthKit**. Obě integrace umožňují číst data naměřená třetími stranami (například krokoměry, spánkové trackery nebo sportovní hodinky) a importovat je do aplikace LifeMeter, případně exportovat záznamy vytvořené v aplikaci zpět do systémového zdravotního úložiště.

Interní databáze potravin

Referenční číselník potravin vychází z veřejně dostupné databáze **FoodData Central** [3], kterou spravuje Ministerstvo zemědělství Spojených států amerických (USDA) prostřednictvím Agricultural Research Service. Databáze FoodData Central je šířena jako veřejná doména pod licencí CC0 1.0 a obsahuje nutriční profily tisíců potravin a ingrediencí včetně kalorické hodnoty, makroživin (bílkoviny, sacharidy, tuky, vláknina) a mikronutrientů.

Datová sada byla pro potřeby aplikace LifeMeter upravena: byly odstraněny záznamy s neúplnými nutričními hodnotami, provedena normalizace jednotek a datových typů do schématu serverové databáze. Upravená sada je uložena přímo v PostgreSQL databázi na serveru a je přístupná prostřednictvím vyhledávacího endpointu REST API. Klient odesílá vyhledávací dotaz na server, který vrátí odpovídající záznamy včetně nutričních hodnot. Výsledky nejsou lokálně cachovány v plném rozsahu; uchovávají se pouze záznamy naposledy použitých položek pro urychlení opakovaného přístupu.

2.3 Use Case diagram

2.3.1 Aktéři

2.3.2 Případy užití

Správa uživatelského účtu

Záznam fyzických aktivit

Sledování výživy

Sledování spánku

Notifikace

Offline režim a synchronizace

Integrace s externími aplikacemi

2.4 Diagram tříd

2.4.1 Popis klíčových tříd

Servisní třídy (Model)

Kontexty (Controller)

Sdílené typy a rozhraní (Types)

2.5 Datový model

2.5.1 Schéma databáze

2.5.2 Popis tabulek

Tabulka `user`

Tabulka `session`

Tabulka `user_profile`

Tabulka `sleep_session`

Tabulka `food_item`

Tabulka `user_meal`

Tabulka `workout`

Tabulka `weight_record`

Tabulka `goal`

3. Metodika a vlastní řešení

3.1 Mobilní klient

3.1.1 Knihovna pro zdravotní data (`health.ts`)

3.1.2 Offline fronta požadavků

3.1.3 Autentizace a perzistence stavu

3.1.4 Notifikace

3.2 Backend API

3.2.1 Technologie a nasazení

3.2.2 CI/CD pipeline

3.2.3 Dokumentace endpointů

Metoda	Cesta	Popis
POST	`/api/auth/sign-up`	Registrace nového uživatele
POST	`/api/auth/sign-in`	Přihlášení uživatele
GET	`/api/health/steps`	Načtení kroků za zvolené období
POST	`/api/health/sync`	Synchronizace zdravotních dat

4. Výsledky

4.1 Funkční části aplikace

4.1.1 Mobilní klient

4.1.2 Backend

4.2 Testování

4.2.1 Testování na fyzickém zařízení

4.2.2 Synchronizace zdravotních dat

5. Diskuse

5.1 Zhodnocení architektonických voleb

5.1.1 React Native vs. Flutter

5.1.2 Vlastní backend vs. Supabase

5.2 Otevřené problémy

5.2.1 30denní limit Health Connect

5.3 Možná rozšíření

Závěr

5.4 Shrnutí

5.5 Podmínky provozu

5.6 Budoucí rozvoj

Literatura

[1] Daniel Torres. *The Real Architecture inside MVC — The beginning.* [Online; cit. 2026-03-11]. Medium, srp. 2021. URL: `https://danstorre.medium.com/the-real-architecture-inside-mvc-the-beginning-50f88af8e0f8` (cit. 11. 03. 2026).

[2] Android Developers. *Build an offline-first app.* [Online; cit. 2026-03-11]. Google, břez. 2026. URL: `https://developer.android.com/topic/architecture/data-layer/offline-first` (cit. 11. 03. 2026).

[3] U.S. Department of Agriculture, Agricultural Research Service, Beltsville Human Nutrition Research Center. *FoodData Central.* [Online; cit. 2026-03-11]. Data ve veřejné doméně, licence CC0 1.0. U.S. Department of Agriculture, 2024. URL: `https://fdc.nal.usda.gov/` (cit. 11. 03. 2026).

Seznam obrázků

2.1 Schéma architektonického vzoru MVC aplikace LifeMeter 10
2.2 Komunikace mezi klientem a serverem aplikace LifeMeter 12
2.3 Přehled komponent systému LifeMeter 12

Seznam tabulek

Seznam příloh

Příloha A: Instalační příručka (lokální prostředí)
Příloha B: Instalační příručka (nasazení na server)
Příloha C: Kompletní dokumentace API endpointů